

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 7
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
Босак В.Ю.
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями): 1)

Изложение цели работы.

2) Задание по лабораторной работе с описанием своего варианта.

3) Спецификации ввода-вывода программы.

4) Текст программы (кратко).

5) Выводы по проделанной работе.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
 - Организовать ввод параметров для создания объектов (использовать экранные компоненты);
 - Осуществить визуализацию графических примитивов
- 8) Задать движение по экрану строк (одна за другой) из списка строк. Направление движения по форме и значение каждой строки выбираются случайным образом.

Задание 2. Реализовать построение заданного типа фрактала по варианту
Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала 8) Кривая дракона

Код:

```
"""Rotating triangle module."""
```

```
# pylint: disable=too-many-arguments, too-many-instance-attributes
# pylint: disable=too-few-public-methods, duplicate-code
# pylint: disable=too-many-positional-arguments
```

```
import tkinter as tk
from tkinter import ttk
import math
import time
from PIL import ImageGrab
```

```
class Triangle:
```

```
    """Triangle class with rotation around center of mass."""
```

```
    def __init__(self, p1, p2, p3):
        self.vertices = [p1, p2, p3]
        self.angle = 0
        self._update_center()
```

```
    def _update_center(self):
        """Calculate center of mass (centroid)."""
```

```

        cx = sum(v[0] for v in self.vertices) / 3
        cy = sum(v[1] for v in self.vertices) / 3
        self.center = (cx, cy)

    def rotate(self, degrees):
        """Rotate triangle by given degrees."""
        self.angle += degrees
        rad = math.radians(degrees)
        cos_a = math.cos(rad)
        sin_a = math.sin(rad)
        cx, cy = self.center

        rotated = []
        for x, y in self.vertices:
            dx = x - cx
            dy = y - cy
            new_x = cx + dx * cos_a - dy * sin_a
            new_y = cy + dx * sin_a + dy * cos_a
            rotated.append((new_x, new_y))
        self.vertices = rotated

    def get_vertices(self):
        """Get current vertices."""
        return self.vertices
task2:
"""Sierpinski carpet fractal module."""

# pylint: disable=too-many-arguments, too-many-instance-attributes
# pylint: disable=too-few-public-methods, duplicate-code
# pylint: disable=too-many-positional-arguments

import tkinter as tk
from tkinter import ttk
import time
from PIL import ImageGrab

class SierpinskiCarpet:
    """Sierpinski carpet fractal generator."""

    def __init__(self, size, depth):
        self.size = size
        self.depth = depth

    def generate(self):
        """Generate fractal pattern."""
        pixels = [[1] * self.size for _ in range(self.size)]
        self._fill(pixels, 0, 0, self.size, self.depth)
        return pixels

    def _fill(self, pixels, x, y, cur_size, depth):

```

```

        """Recursively fill holes."""
        if depth == 0 or cur_size < 3:
            return

        new_size = cur_size // 3
        if new_size == 0:
            return

        cx = x + new_size
        cy = y + new_size
        for i in range(cy, cy + new_size):
            for j in range(cx, cx + new_size):
                if 0 <= i < self.size and 0 <= j < self.size:
                    pixels[i][j] = 0

        for row in range(3):
            for col in range(3):
                if row == 1 and col == 1:
                    continue
                self._fill(
                    pixels,
                    x + col * new_size,
                    y + row * new_size,
                    new_size,
                    depth - 1
                )

class CarpetApp:
    """Main application class."""

    def __init__(self, parent):
        self.parent = parent
        self.parent.title("Ковёр Серпинского")
        self.parent.geometry("900x800")
        self.parent.configure(bg="#2c3e50")

        self.depth = 3
        self.size = 600
        self.after_id = None
        self.is_animating = False

        self._create_controls()
        self._create_canvas()
        self._draw_fractal()

```

Вывод: освоены возможности языка программирования Python в разработке оконных приложений.